

tf.compat.v1.py_func Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/py_func

Wraps a python function and uses it as a TensorFlow op.

Function Signatures

```
tf.compat.v1.py_func( func, inp, Tout, stateful=True,  
name=None )
```

Code Examples

```
tf.compat.v1.py_func(  
    func, inp, Tout, stateful=True, name=None  
)
```

```
tf.compat.v1.py_func(  
    func, inp, Tout, stateful=True, name=None  
)
```

```
def fn_using_numpy(x):  
    x[0] = 0.  
    return x  
tf.compat.v1.py_func(fn_using_numpy, inp=[tf.constant([1., 2.])],  
    Tout=tf.float32, stateful=False)
```

```
def fn_using_numpy(x):
```

```
tf.compat.v1.py_func(fn_using_numpy, inp=[tf.constant([1., 2.])],
```

```
Tout=tf.float32, stateful=False)
```

Documentation

Wraps a python function and uses it as a TensorFlow op.

Migrate to TF2

This name was deprecated and removed in TF2, but `tf.numpy_function` is a near-exact replacement, just drop the `stateful` argument (all

`tf.numpy_function` calls are considered stateful). It is compatible with eager execution and `tf.function`.

`tf.py_function` is a close but not an exact replacement, passing TensorFlow tensors to the wrapped function instead of NumPy arrays, which provides gradients and can take advantage of accelerators.

Before:

After:

Description

Used in the notebooks

- TensorFlow Probability Case Study: Covariance Estimation

Given a python function `func`, which takes numpy arrays as its arguments and returns numpy arrays as its outputs, wrap this function as an operation in a TensorFlow graph. The following snippet constructs a simple TensorFlow graph that invokes the `np.sinh()` NumPy function as a operation in the graph:

- The body of the function (i.e. `func`) will not be serialized in a

`GraphDef`. Therefore, you should not use this function if you need to serialize your model and restore it in a different environment.

- The operation must run in the same address space as the Python program

that calls `tf.compat.v1.py_func()`. If you are using distributed TensorFlow, you must run a `tf.distribute.Server` in the same process as the program that calls `tf.compat.v1.py_func()` and you must pin the created operation to a device in that server (e.g. using with `tf.device(:)`).

The body of the function (i.e. `func`) will not be serialized in a `GraphDef`. Therefore, you should not use this function if you need to serialize your model and restore it in a different environment.

The operation must run in the same address space as the Python program that calls `tf.compat.v1.py_func()`. If you are using distributed TensorFlow, you must run a `tf.distribute.Server` in the same process as the program that calls `tf.compat.v1.py_func()` and you must pin the created operation to a device in that server (e.g. using with `tf.device(:)`).

Returns